



IDEGEN PROCESSOR

CS-222 (A) PROJECT

GROUP MEMBERS:

AHMAD HASAN	2019041
TAIMOOR AHMAD	2019511
ALI ABDUL RAZZAQ	2019058
AHMAD MAAZ	2019043
MUTEEB UR REHMAN	2019419

SUBMISSION DATE: 31ST MAY 2021.

SUBMITTED TO: DR. GHULAM ABBAS.

Contents

Abstract:	2
Introduction:	3
Instruction format and architecture:	3
Memory:	4
Addressing modes:	4
Direct addressing mode:	5
Indirect addressing mode:	5
Immediate mode:	6
Indexed addressing mode:	6
Types of instructions	7
Memory Reference Instruction:	7
Register Reference Instruction:	7
Input/Output Instructions:	8
Registers	9
Flip flops/ Flags	11
Bus Design	12
Instruction Set and Control Signals	13
Memory reference instructions	13
Memory Reference Control Signals	16
Register reference instructions	22
Register Reference control signals	22
Input output instructions	25
Input/Output control signals	25
ALU	26
Control unit	27
Fetch decode cycle	28
Interrupt Cycle	29
Control of Register	31
Control of Flag	32
Control of Common Bus	32
Complete computer description	33
ALU Circuits	34
Bibliography	37

Abstract:

The basic computer that we have designed has 19 bits for one complete instruction (also known as a word). The whole instruction is divided into 3 parts, one that specifies the addressing mode, while the other 2 designate the opcode and operand. The size of memory is $2^{12}=4096$, of which 511 words are reserved for the memory of the stack, with the size of each word being equal to 19 bits.

We have two bits dedicated for the addressing mode which means we have a total of 4 addressing modes: direct addressing, indirect addressing, immediate addressing, and indexed addressing.

Our 5 bits opcode enable us to have 32 unique instructions (00000-11111) which, when combined with the addressing mode bits, provide us with numerous possibilities.

There are 3 types of instructions for our processor; memory reference instruction, register reference instruction and I/O instructions.

Registers used for our processor include the ALU, a multiplier (not a register), a PC, an AR, a DR, an Index Register allowing us a whole new addressing mode, TR (temporary register), a stack pointer register pointing to the top of the stack, an AC, an Input Register, and an Output Register. Furthermore, an Input/output cycle, along with an interrupt cycle are designed according to our addressing modes.

Introduction:

Instruction format and architecture:

An Instruction format suggests how the code is read as a machine code when converted from an assembly language.

In the case of our processor, the instruction format is as follows:



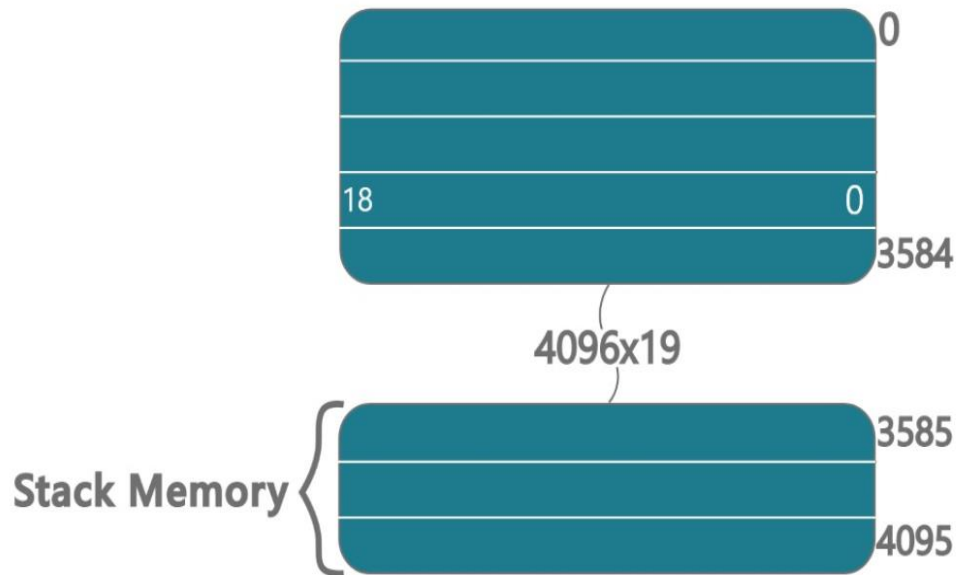
An instruction of our 5-bit processor has 3 parts, one that specifies the addressing mode, and the latter 2 that specify the opcode and address. The 2 addressing mode bits are the most significant bits (left most), followed by a 5-bit opcode, while the operand has a total of 12 bits.

- An Addressing mode: Lets the processor know how the operand of an instruction is specified.
- An Opcode: A distinct combination of bits that form the instruction that is to be executed on the operand.
- An Operand: This contains data on which the instruction is to be executed upon.

The 19-bit instruction size enables us to have a large set of instructions. These instruction sets are unique to every processor and vary according to the total number of bits available for each instruction and the registers available for the processor. These instructions are represented as an assembly language code.

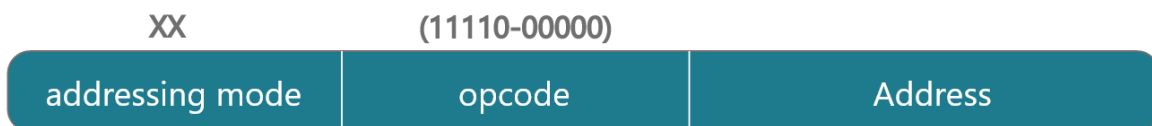
Memory:

The memory size depends on the size of the instruction. The size of our processor's memory is $2^{12}=4096$ and 19 bits are used for each individual instruction.



Addressing modes:

The instruction format specifies 2 bits for the addressing modes, enabling us to have 4 distinct combinations of bits and thus we obtain 4 different addressing modes. These bits play a vital role in memory reference instructions.

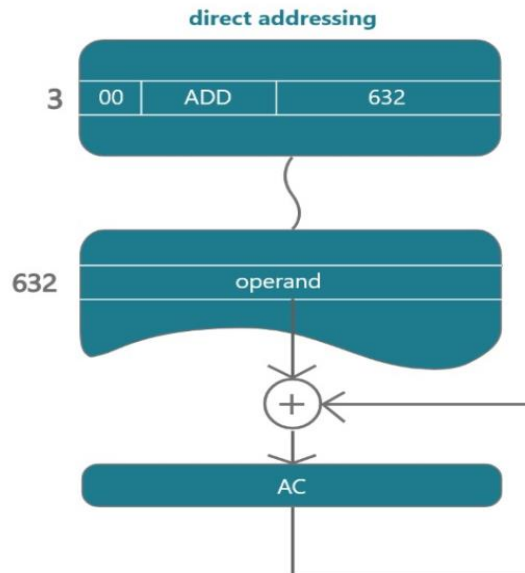


The XX for the addressing mode range from 00 to 11.

- Direct addressing mode: XX = 00
- Indirect addressing mode: XX=01
- Immediate addressing: XX=10
- Indexed addressing: XX=11

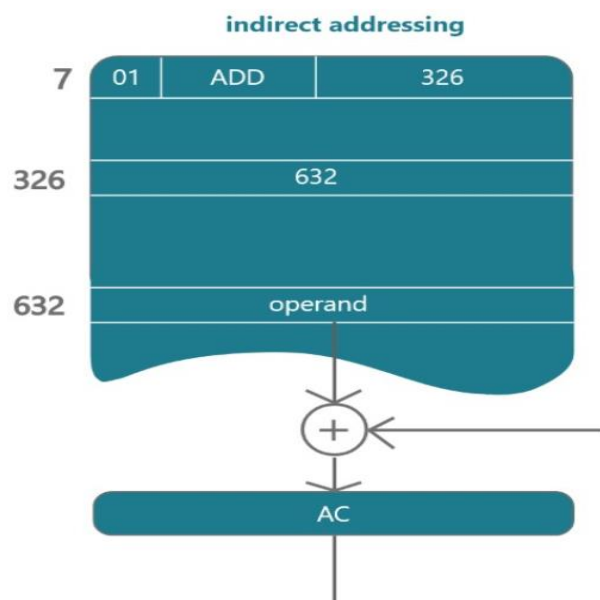
Direct addressing mode:

In this mode, the operand has the address which contains the data over which the opcode is to be executed.



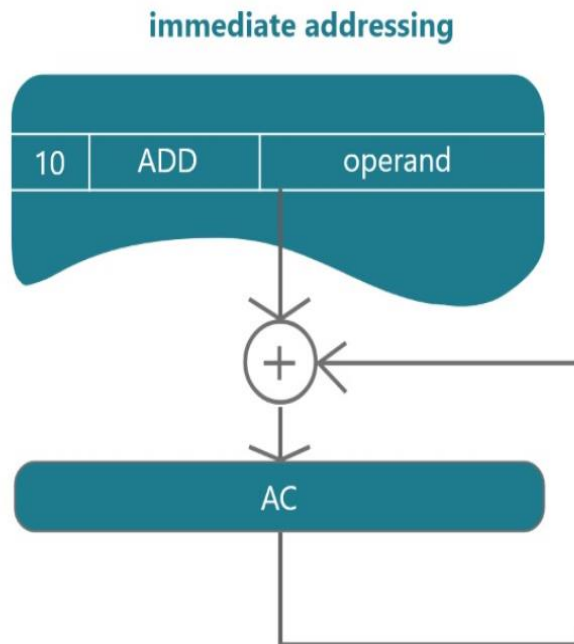
Indirect addressing mode:

In this mode, an operand has the address that points to the address containing the data over which the opcode is to be executed.



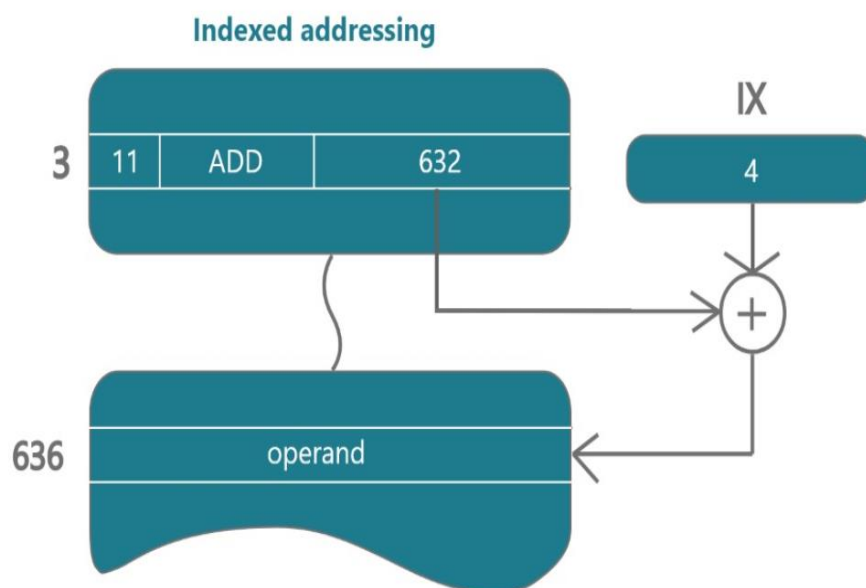
Immediate mode:

In this mode, the operand contains the data over which the opcode is to be executed.



Indexed addressing mode:

In this addressing mode, an effective address is used, which is the address in the operand offset with the data in the index register. This effective address is the address which contains the data over which the opcode is to be executed.



Types of instructions

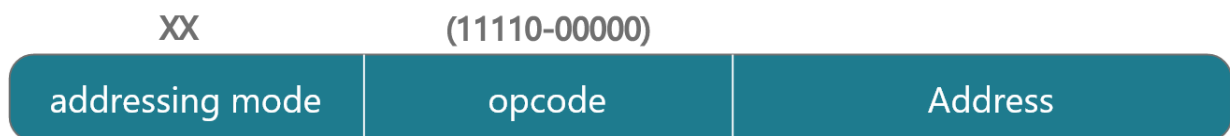
Memory Reference Instruction:

As stated in the name itself, Memory Reference instructions involve the memory of the computer. These instructions indirectly make use of the operand as they use the addressing part of the instruction, which then refers to the operand. This depends on the type of addressing mode being used.

These instructions using the operand from memory, perform a specific operation depending upon the type of instruction.

The addressing part of the memory reference instructions is reserved for the address of the operands. For the Opcodes we have 32 different combinations varying from 00000 ~ 11111 hence we can add 31 different memory reference instructions at maximum.

Every single memory reference instruction has a 2-bit addressing mode, indicating the addressing mode of a particular instruction. As there are 2 bits, hence there are ($2^2=4$) 4 addressing modes that can be used for an instruction.



Register Reference Instruction:

These instructions do not require memory and so there is no need for the address to refer to the operand. All operations of the register reference instructions are concerned with the registers of the computer.

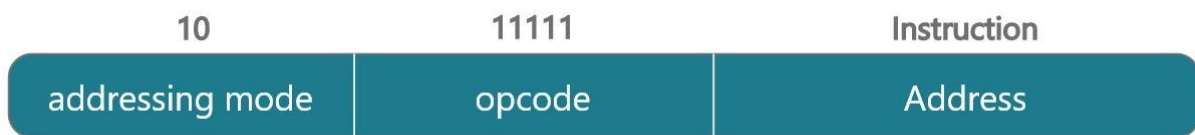
Register Reference instructions have a fixed addressing mode for each individual instruction for i.e., 00. Similarly, The Opcode of each Register Reference instructions is fixed to 11111 and only the addressing part of an instruction varies. This variation depends upon the instruction and its corresponding register operation.



Input/Output Instructions:

A typical processor can also possess one or more external devices. In order to communicate with these external devices, input and output instructions are used.

The addressing mode in case of Input and Output instructions is 01 and opcode is also fixed to 11111. Only the addressing part in an instruction is changed and for each instruction, there is an Input / Output operation.



Registers

- **DR:**

Data register or memory buffer register is used either to contain the contents transferred from some memory word or contains the contents to be transferred to the memory word.



- **AR:**

Memory address register contains the current instruction's address that must be either fetched from or to the memory.



- **AC:**

The results from the arithmetic logic unit are temporarily stored into the accumulator.



- **IR:**

Current instruction register, as the name suggests, holds the address of the instruction that is being executed at the instance.



- **PC:**

Program counter holds the address of the next instruction that is to be executed and increments to the next instruction every time the contents of address at the AR are moved to the DR.



- **TR:**

Intermediate results of certain operations or calculations are sometimes stored in the temporary register.



- **INPR:**

Data from an input device must be stored first. For this purpose of storage, an input register is used.



- **OUTR:**

Data stored in the output registers is eventually sent to an output device.



- **IX:**

An Index register holds the offset value for the address in the instruction.



- **SP:**
This stack pointer holds the address of the top of the stack.



Flip flops/ Flags

I₁: A flip flop that stores a bit which is part of the addressing mode and is further transferred to combinational logic.

I₂: Another flip flop which stores one bit which is part of the addressing mode and is further transferred to combinational logic.

S (Sequence counter): This generates the timing sequence with each instruction.

E (Extended AC):

This is for storing the extra bit which results from any operation i.e addition, multiplication etc.

R (Interrupt Flag):

Stores a bit, 0 or 1, depending upon which cycle to choose for execution. 0 will indicate the start of fetch execute cycle while 1 will start the interrupt cycle.

IEN (Interrupt enable):

This stores a bit, 1 or 0, which determines whether to enable or perform the interrupt. If the IEN has a 1 stored in it, it will consider and perform the incoming interrupt cycle, whereas if there is a 0 stored at the IEN, at that instance, the incoming signal for the interrupt cycle will be ignored.

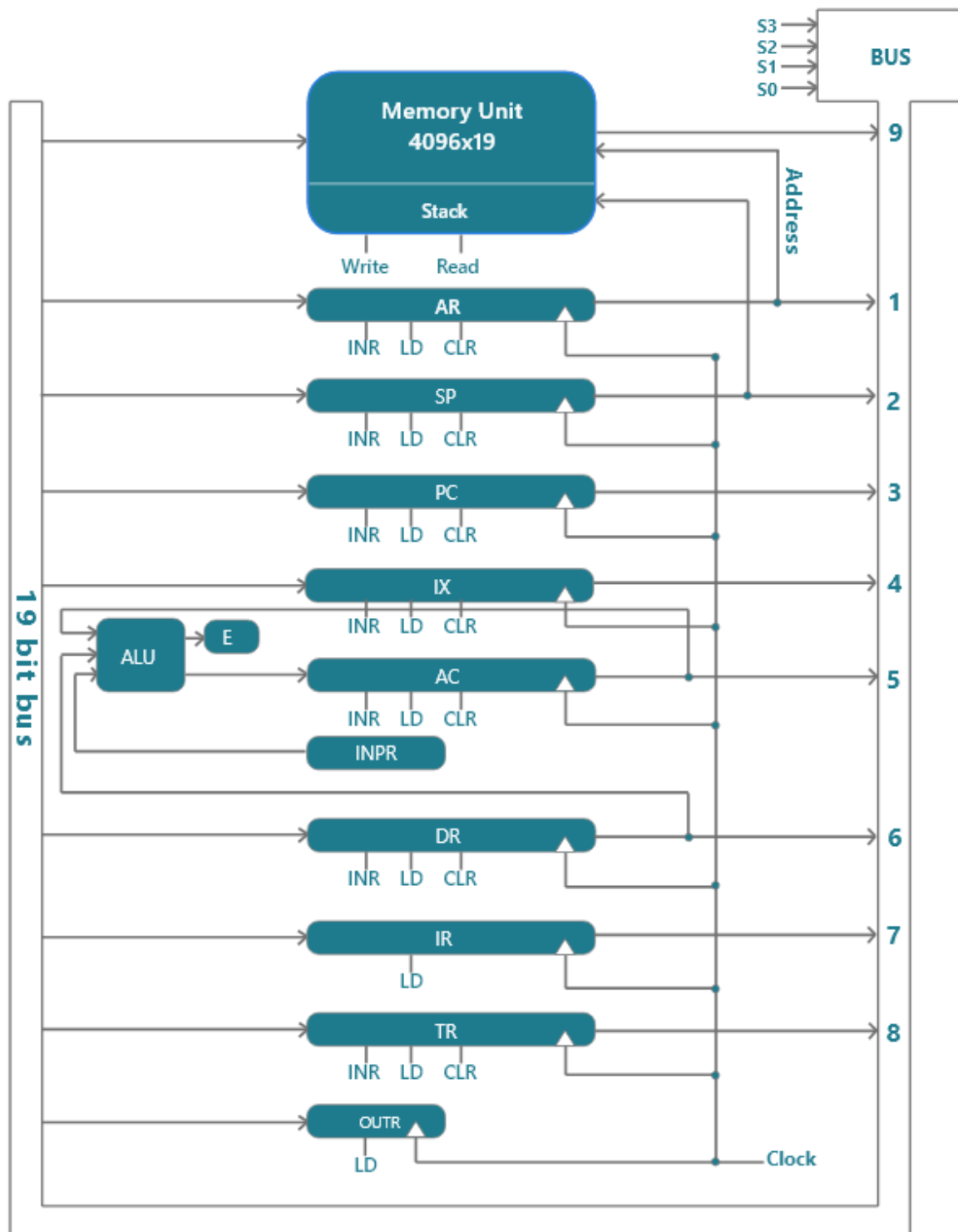
FGI (Input Flag):

This flag determines whether to receive the input from the input device. If it is on, meaning the bit stored in it is 1, it will receive the input and if it is off, meaning that the stored bit in the FGI is now 0, the incoming input will be ignored or nullified.

FGO (Output flag):

This flag determines whether to send the output to the output device. If it is on, meaning the bit stored in it is 1, it will send out the output and if it is off, meaning that the stored bit in the FGO is now 0, the output will be ignored or nullified.

Bus Design



Instruction Set and Control Signals

Memory reference instructions

Symbol	Binary code	HEX CODE	Description
ADD	00 00000 XXXXXXXXXXXXX 01 00000 XXXXXXXXXXXXX 10 00000 XXXXXXXXXXXXX 11 00000 XXXXXXXXXXXXX	00 XXX 20 XXX 40 XXX 60 XXX	Add memory word to AC
ADX	00 00001 XXXXXXXXXXXXX 01 00001 XXXXXXXXXXXXX 10 00001 XXXXXXXXXXXXX 11 00001 XXXXXXXXXXXXX	01 XXX 21 XXX 41 XXX 61 XXX	Add memory word to IX
ADC	00 00010 XXXXXXXXXXXXX 01 00010 XXXXXXXXXXXXX 10 00010 XXXXXXXXXXXXX 11 00010 XXXXXXXXXXXXX	02 XXX 22 XXX 42 XXX 62 XXX	Add with carry memory word to AC
MINAC	00 00011 XXXXXXXXXXXXX 01 00011 XXXXXXXXXXXXX 10 00011 XXXXXXXXXXXXX 11 00011 XXXXXXXXXXXXX	03 XXX 23 XXX 43 XXX 63 XXX	Subtract memory word to AC
SUBAC	00 00100 XXXXXXXXXXXXX 01 00100 XXXXXXXXXXXXX 10 00100 XXXXXXXXXXXXX 11 00100 XXXXXXXXXXXXX	04 XXX 24 XXX 44 XXX 64 XXX	Subtract with borrow memory word to AC
MUAC	00 00101 XXXXXXXXXXXXX 01 00101 XXXXXXXXXXXXX 10 00101 XXXXXXXXXXXXX 11 00101 XXXXXXXXXXXXX	05 XXX 25 XXX 45 XXX 65 XXX	Multiply memory word to AC
MULTIPIX	00 00110 XXXXXXXXXXXXX 01 00110 XXXXXXXXXXXXX 10 00110 XXXXXXXXXXXXX 11 00110 XXXXXXXXXXXXX	06 XXX 26 XXX 46 XXX 66 XXX	Multiply memory word to IX
DIVAC	00 00111 XXXXXXXXXXXXX 01 00111 XXXXXXXXXXXXX 10 00111 XXXXXXXXXXXXX 11 00111 XXXXXXXXXXXXX	07 XXX 27 XXX 47 XXX 67 XXX	Divide memory word to AC
DIVISIX	00 01000 XXXXXXXXXXXXX 01 01000 XXXXXXXXXXXXX 10 01000 XXXXXXXXXXXXX 11 01000 XXXXXXXXXXXXX	08 XXX 28 XXX 48 XXX 68 XXX	divide memory word to IX

AND	00 01001 XXXXXXXXXXXXX 01 01001 XXXXXXXXXXXXX 10 01001 XXXXXXXXXXXXX 11 01001 XXXXXXXXXXXXX	09 XXX 29 XXX 49 XXX 69 XXX	AND memory word to AC
XORAC	00 01010 XXXXXXXXXXXXX 01 01010 XXXXXXXXXXXXX 10 01010 XXXXXXXXXXXXX 11 01010 XXXXXXXXXXXXX	0A XXX 2A XXX 4A XXX 6A XXX	XOR memory word to AC
ORAC	00 01011 XXXXXXXXXXXXX 01 01011 XXXXXXXXXXXXX 10 01011 XXXXXXXXXXXXX 11 01011 XXXXXXXXXXXXX	0B XXX 2B XXX 4B XXX 6B XXX	OR memory word to AC
NOTAC	00 01100 XXXXXXXXXXXXX 01 01100 XXXXXXXXXXXXX 10 01100 XXXXXXXXXXXXX 11 01100 XXXXXXXXXXXXX	0C XXX 2C XXX 4C XXX 6C XXX	NOT memory word and store in AC
DECSZ	00 01101 XXXXXXXXXXXXX 01 01101 XXXXXXXXXXXXX 11 01101 XXXXXXXXXXXXX	0D XXX 2D XXX 6D XXX	Decrement skip if zero
ISZ	00 01110 XXXXXXXXXXXXX 01 01110 XXXXXXXXXXXXX 11 01110 XXXXXXXXXXXXX	0E XXX 2E XXX 6E XXX	Increment skip if zero
PUSHAC	00 01111 XXXXXXXXXXXXX 01 01111 XXXXXXXXXXXXX 10 01111 XXXXXXXXXXXXX 11 01111 XXXXXXXXXXXXX	0F XXX 2F XXX 4F XXX 6F XXX	Push AC to stack memory
POPAC	00 10000 XXXXXXXXXXXXX 01 10000 XXXXXXXXXXXXX 11 10000 XXXXXXXXXXXXX	10 XXX 30 XXX 70 XXX	Pop from stack memory to AC
SPACM	00 10001 XXXXXXXXXXXXX 01 10001 XXXXXXXXXXXXX 11 10001 XXXXXXXXXXXXX	11 XXX 31 XXX 71 XXX	Swap AC and memory word
GCTB	00 10010 XXXXXXXXXXXXX 01 10010 XXXXXXXXXXXXX 10 10010 XXXXXXXXXXXXX 11 10010 XXXXXXXXXXXXX	12 XXX 32 XXX 52 XXX 72 XXX	Convert Gray code to Binary
LDA	00 10011 XXXXXXXXXXXXX 01 10011 XXXXXXXXXXXXX 10 10011 XXXXXXXXXXXXX 11 10011 XXXXXXXXXXXXX	13 XXX 33 XXX 53 XXX 73 XXX	Convert Binary to Gray Code

LDA	00 10100 XXXXXXXXXXXXX 01 10100 XXXXXXXXXXXXX 10 10100 XXXXXXXXXXXXX 11 10100 XXXXXXXXXXXXX	14 XXX 34 XXX 54 XXX 74 XXX	Load AC from memory
ACDLT	00 10101 XXXXXXXXXXXXX 01 10101 XXXXXXXXXXXXX 11 10101 XXXXXXXXXXXXX	15 XXX 35 XXX 75 XXX	If AC is less than DR then jump to address
BUN	00 10110 XXXXXXXXXXXXX 01 10110 XXXXXXXXXXXXX 11 10110 XXXXXXXXXXXXX	16 XXX 36 XXX 76 XXX	Branch unconditionally
BSA	00 10111 XXXXXXXXXXXXX 01 10111 XXXXXXXXXXXXX 11 10111 XXXXXXXXXXXXX	17 XXX 37 XXX 77 XXX	Branch and save return address
STA	00 11000 XXXXXXXXXXXXX 01 11000 XXXXXXXXXXXXX 11 11000 XXXXXXXXXXXXX	18 XXX 38 XXX 78 XXX	Store AC to memory
STIX	00 11001 XXXXXXXXXXXXX 01 11001 XXXXXXXXXXXXX 11 11001 XXXXXXXXXXXXX	19 XXX 39 XXX 79 XXX	Store IX to memory
COMPAC	00 11010 XXXXXXXXXXXXX 01 11010 XXXXXXXXXXXXX 10 11010 XXXXXXXXXXXXX 11 11010 XXXXXXXXXXXXX	1A XXX 3A XXX 5A XXX 7A XXX	COMPARE THE CONTENT OF AC WITH THE GIVEN ADDRESS
COMPIX	00 11011 XXXXXXXXXXXXX 01 11011 XXXXXXXXXXXXX 10 11011 XXXXXXXXXXXXX 11 11011 XXXXXXXXXXXXX	1B XXX 3B XXX 5B XXX 7B XXX	COMPARE THE CONTENT OF IX WITH THE GIVEN ADDRESS
JUMPE	00 11100 XXXXXXXXXXXXX 01 11100 XXXXXXXXXXXXX 11 11100 XXXXXXXXXXXXX	1C XXX 3C XXX 7C XXX	Following a compare instruction, jump to address if compare was true
JUMPN	00 11101 XXXXXXXXXXXXX 01 11101 XXXXXXXXXXXXX 11 11101 XXXXXXXXXXXXX	1D XXX 3D XXX 7D XXX	Following a compare instruction, jump to address if compare was false
MINIX	00 11110 XXXXXXXXXXXXX 01 11110 XXXXXXXXXXXXX 10 11110 XXXXXXXXXXXXX 11 11110 XXXXXXXXXXXXX	1E XXX 3E XXX 5E XXX 7E XXX	Subtract memory word to IX

Memory Reference Control Signals

1) ADD

This instruction adds the memory word (stored in the address provided) to AC and then stores the result in the AC.

$DR \leftarrow M[AR]$	D_0T_5
$AC \leftarrow AC + DR, E \leftarrow 0, S \leftarrow 0$	D_0T_6

2) ADX

This instruction adds the memory word (stored in the address provided) to IX and then stores the result in the IX.

$DR \leftarrow M[AR]$	D_1T_5
$IX \leftarrow IX + DR, E \leftarrow 0, S \leftarrow 0$	D_1T_6

3)

ADC

This instruction adds the memory word (stored in the address provided) to AC along with a carry '1' and then stores the result in the AC.

$AC \leftarrow 1$	D_2T_5
$DR \leftarrow M[AR]$	D_2T_6
$AC \leftarrow AC + DR, E \leftarrow 0, S \leftarrow 0$	D_2T_7

4) MINAC

This instruction takes the difference between the memory word (stored in the address provided) and the AC and then stores the result in the AC.

$DR \leftarrow M[AR]$	D_3T_5
$DR \leftarrow DR' + 1$	D_3T_6
$AC \leftarrow AC + DR, S \leftarrow 0$	D_3T_7

5) SUBAC

This instruction takes the difference between the memory word (stored in the address provided) and the AC using borrow and then stores the result in the AC.

$DR \leftarrow M[AR]$	D_4T_5
$DR \leftarrow DR'$	D_4T_6
$AC \leftarrow AC + DR, S \leftarrow 0$	D_4T_7

6) MUAC

This instruction multiplies the memory word with the AC and then stores the result in the AC.

$DR \leftarrow M[AR]$	D_5T_5
$AC \leftarrow AC * DR, S \leftarrow 0$	D_5T_6

7) MULTIPIX

This instruction multiplies the memory word with the AC and then stores the result in the IX.

$DR \leftarrow M[AR]$	D_6T_5
$IX \leftarrow IX * DR, S \leftarrow 0$	D_6T_6

8) DIVAC

This instruction divides the memory word with the AC and then stores the result in the AC.

$DR \leftarrow M[AR]$	D_7T_5
$AC \leftarrow AC / DR, S \leftarrow 0$	D_7T_6

9) DIVISIX

This instruction divides the memory word with the AC and then stores the result in the IX.

$DR \leftarrow M[AR]$	D_8T_5
$IX \leftarrow AC / DR, S \leftarrow 0$	D_8T_6

10) AND

This instruction takes the memory word and ANDs it with the AC and then stores the result in the AC.

$DR \leftarrow M[AR]$	D_9T_5
$AC \leftarrow AC \wedge DR, S \leftarrow 0$	D_9T_6

11)XORAC

This instruction takes the memory word and XORs it with the AC and then stores the result in the AC.

$DR \leftarrow M[AR]$	$D_{10}T_5$
$AC \leftarrow AC \oplus DR, S \leftarrow 0$	$D_{10}T_6$

12)ORAC

This instruction takes the memory word and ORs it with the AC and then stores the result in the AC.

$DR \leftarrow M[AR]$	$D_{11}T_5$
$AC \leftarrow AC \vee DR, S \leftarrow 0$	$D_{11}T_6$

13)NOTAC

This instruction takes the memory word and ORs it with the AC and then stores the result in the AC.

$DR \leftarrow M[AR]$	$D_{12}T_5$
$AC \leftarrow DR', S \leftarrow 0$	$D_{12}T_6$

14)DECSZ

This instruction skips the next instruction if when address is decremented and is equal to zero.

$M[AR] \leftarrow M[AR] + 1$	$D_{13}T_5$
If $M[AR] - 1 = 0$ then $PC \leftarrow PC+1$	$D_{13}T_6$

15)ISZ

This instruction skips the next instruction if when address is incremented and is equal to zero.

$M[AR] \leftarrow M[AR] + 1$	$D_{14}T_5$
If $M[AR] + 1 = 0$ then $PC \leftarrow PC+1$	$D_{14}T_6$

16)PUSHAC

This instruction pushes the content in DR on top of the stack in memory.

$SP \leftarrow SP - 1$	$D_{15}T_5$
$M[SP] \leftarrow DR$	$D_{15}T_6$

17) POPAC

This instruction pops the content from the top of the memory and transfers it to DR.

$DR \leftarrow M[SP]$	$D_{16}T_5$
$SP \leftarrow SP + 1$	$D_{16}T_6$

18) SPACM

This instruction swaps the contents of AC and DR.

$DR \leftarrow M[AR]$	$D_{17}T_5$
$TR \leftarrow AC$	$D_{17}T_6$
$AC \leftarrow DR$	$D_{17}T_7$
$DR \leftarrow TR$	$D_{17}T_8$
$TR \leftarrow 0, S \leftarrow 0$	$D_{17}T_9$

19) GCTB

This instruction stores the content of memory address to DR. Then the content of DR is converted to binary from grey code, which is then stored into the AC.

$DR \leftarrow M[AR]$	$D_{18}T_5$
$AC \leftarrow DR^{\sim}$	$D_{18}T_6$

20) BTGC

This instruction stores the content of memory address to DR. Then the content of DR is converted to gray code from binary, which is then stored into the AC.

$DR \leftarrow M[AR]$	$D_{19}T_5$
$AC \leftarrow DR^{\sim}$	$D_{19}T_6$

21) LDA

This instruction loads the AC with the data that is at the address specified.

$AC \leftarrow M[AR], S \leftarrow 0$	$D_{20}T_5$
---------------------------------------	-------------

22)ACDLT

If the content of AC is less than the content of DR, then jump to the given address.

$DR \leftarrow M[AR]$	$D_{21}T_5$
If $AC < DR$ then $PC \leftarrow AR, S \leftarrow 0$	$D_{21}T_6$

23)BUN

This instruction branches to another instruction without saving the return address of the current instruction.

$PC \leftarrow AR, S \leftarrow 0$	$D_{22}T_5$
------------------------------------	-------------

24)BSA

This instruction branches to another instruction after saving the return address of the current instruction.

$M[AR] \leftarrow PC, AR \leftarrow AR + 1$	$D_{23}T_5$
$PC \leftarrow AR, S \leftarrow 0$	$D_{23}T_6$

25)STA

This instruction stores the content of the AC to a specified address.

$M[AR] \leftarrow AC, S \leftarrow 0$	$D_{24}T_5$
---------------------------------------	-------------

26)STIX

This instruction stores the content of the IX to a specified address.

$M[AR] \leftarrow IX, S \leftarrow 0$	$D_{25}T_5$
---------------------------------------	-------------

27)COMPAC

Compare the content of the AC with the content of the given address.

$DR \leftarrow M[AR]$	$D_{26}T_5$
If $AC = DR$ then $TR(0) \leftarrow 1$ else $TR(0) \leftarrow 1, S \leftarrow 0$	$D_{26}T_6$

28) COMPIX

Compare the content of the IX with the content of given address

$DR \leftarrow M[AR]$	$D_{27}T_5$
If $IX = DR$ then $TR(0) \leftarrow 1$ else $TR(0) \leftarrow 1, S \leftarrow 0$	$D_{27}T_6$

29) JUMPE

This instruction works with COMPAC and COMPIX. If COMPAC or COMPIX returns true, then jump to the given address.

If $TR(0) = 1$ then $PC \leftarrow AR, S \leftarrow 0$	$D_{28}T_5$
--	-------------

30) JUMPN

This instruction works with COMPAC and COMPIX. If COMPAC or COMPIX returns false, then jump to the given address.

If $TR(0) = 0$ then $PC \leftarrow AR, S \leftarrow 0$	$D_{29}T_5$
--	-------------

31) MINIX

This instruction takes the difference between the memory word (stored in the address provided) and the IX and then stores the result in the IX.

$DR \leftarrow M[AR]$	D_3T_5
$DR \leftarrow DR' + 1$	D_3T_6
$IX \leftarrow IX + DR, S \leftarrow 0$	D_3T_7

Register reference instructions

Symbol	Binary Code	HEX CODE	Description
TRANF	00 11111 000000000000	1F 000	Transfer from memory
DCA	00 11111 000000000001	1F 001	Decrement AC
INC	00 11111 000000000010	1F 002	Increment AC
LEFTS	00 11111 000000000011	1F 003	Left shift
RIGHTS	00 11111 000000000100	1F 004	Right shift
CIL	00 11111 000000000101	1F 005	Rotate left
CIR	00 11111 000000000110	1F 006	Rotate right
CLA	00 11111 000000000111	1F 007	Clear AC
CLE	00 11111 000000001000	1F 008	Clear E
CMA	00 11111 000000001001	1F 009	Complement AC
CME	00 11111 000000001010	1F 00A	Complement E
SNA	00 11111 000000001011	1F 00B	Skip next instruction if AC is negative
SPA	00 11111 000000001100	1F 00C	Skip next instruction if AC is positive
SZA	00 11111 000000001101	1F 00D	Skip next instruction if AC is zero
SZE	00 11111 000000001110	1F 00E	Skip next instruction if E is zero
STP	00 11111 000000001111	1F 00F	Halt

Register Reference control signals

$p = D_{31}l_1'l_2'T_4$ $X = IR(w)$ where $w = 0, 1, 2, 3, \dots, 18$

1) TRF

This instruction transfers the content of AC to the temporary register TR for holding intermediate data.

$TR \leftarrow AC, AC \leftarrow 0, S \leftarrow 0$	pX_0
---	--------

2) DCA

This instruction decrements the data in AC by 1.

$AC \leftarrow AC - 1, S \leftarrow 0$	pX_1
--	--------

3) ICA

This instruction increments the data in AC by 1.

$AC \leftarrow AC + 1, S \leftarrow 0$	pX_2
--	--------

4) LES

This instruction performs a left shift on the content of AC.

$AC \leftarrow \text{shl } AC, S \leftarrow 0$	pX_3
--	--------

5) RIS

This instruction performs a right shift on the content of AC

$AC \leftarrow \text{shr } AC, S \leftarrow 0$	pX_4
--	--------

6) ROL

This instruction performs a circular left shift on the content of AC.

$AC \leftarrow \text{shl } AC, AC(17) \leftarrow E,$ $E \leftarrow AC(0), S \leftarrow 0$	pX_5
--	--------

7) ROR

This instruction performs a circular right shift on the content of AC.

$AC \leftarrow \text{shr } AC, AC(0) \leftarrow E, E \leftarrow AC(15),$ $S \leftarrow 0$	pX_6
--	--------

8) CLRA

This instruction clears the content of AC.

$AC \leftarrow 0, S \leftarrow 0$	pX_7
-----------------------------------	--------

9) CLRE

This instruction clears the content of E.

$E \leftarrow 0, S \leftarrow 0$	pX_8
----------------------------------	--------

10)CMPA

This instruction complements the contents of AC.

$AC \leftarrow AC', S \leftarrow 0$	pX_9
-------------------------------------	--------

11)CMPE

This instruction complements the contents of E.

$E \leftarrow E', S \leftarrow 0$	pX_{10}
-----------------------------------	-----------

12)SAN

This instruction skips the next instruction of the program if the 17th bit of AC is 1 i.e. content of AC is negative.

If $(AC(17) = 1)$ then $(PC \leftarrow PC + 1), S \leftarrow 0$	pX_{11}
---	-----------

13)SAP

This instruction skips the next instruction of the program if the 17th bit of AC is zero i.e., content of AC is positive.

If $(AC(17) = 0)$ then $(PC \leftarrow PC + 1), S \leftarrow 0$	pX_{12}
---	-----------

14)SNAZ

This instruction skips the next instruction of the program if AC is zero.

If $(AC = 0)$ then $(PC \leftarrow PC + 1), S \leftarrow 0$	pX_{13}
---	-----------

15)SNEZ

This instruction skips the next instruction of the program if E is zero.

If $(E = 0)$ then $(PC \leftarrow PC + 1), S \leftarrow 0$	pX_{14}
--	-----------

16)STP

This instruction indicates the end of a program and stops its execution.

$S \leftarrow 0$	pX_{15}
------------------	-----------

Input output instructions

Symbol	Binary Code	HEX CODE	Description
INP	11 11111 000000000000	7F 000	Input data to AC
OUT	11 11111 000000000001	7F 001	Output data from AC
SKI	11 11111 000000000010	7F 002	Skip on input flag
SKO	11 11111 000000000011	7F 003	Skip on output flag
ION	11 11111 000000000100	7F 004	Interrupt on
IOF	11 11111 000000000101	7F 005	Interrupt off

Input/Output control signals

$g = D_{31}I_1'I_2T_4$ $X = IR(w)$ where $w = 0, 1, 2, 3, \dots, 18$

1) INP

This instruction transfers data from input register to AC

$AC(0-11) \leftarrow IR, FGI \leftarrow 0, S \leftarrow 0$	gX_0
--	--------

2) OUT

This instruction transfers data from AC to the output register.

$OUTR \leftarrow AC(0-11), FGO \leftarrow 0, S \leftarrow 0$	gX_1
--	--------

3) SKI

This instruction skips the next instruction if input flag is on.

If $(FGI = 1)$ then $(PC \leftarrow PC + 1), S \leftarrow 0$	gX_2
--	--------

4) SKO

This instruction skips the next instruction if output flag is on.

If $(FGO = 0)$ then $(PC \leftarrow PC + 1), S \leftarrow 0$	bZ_3
--	--------

5) ION

This instruction turns on the interrupt flag.

$IEN \leftarrow 1, S \leftarrow 0$	gX_4
------------------------------------	--------

6) IOF

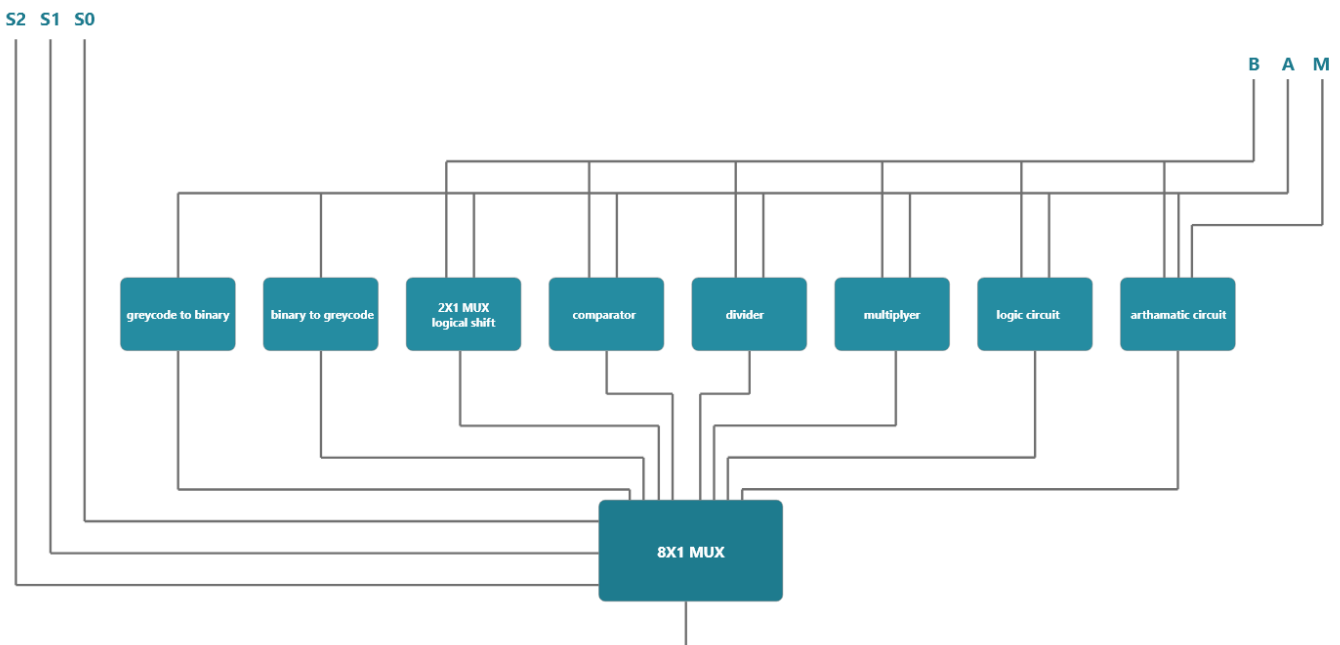
This instruction turns off the interrupt flag.

$IEN \leftarrow 0, S \leftarrow 0$	gX_5
------------------------------------	--------

ALU

Our arithmetic and logic unit constitutes of circuits that are able to perform 8 distinct type operations. The ALU selects which of the operation to choose from using the 3 select lines ($2^3 = 8$). The circuits within the ALU as followed.

1. Arithmetic circuit
2. Logic circuit
3. Multiplier
4. Divider
5. Comparator
6. Logic shift circuit
7. Binary to gray code convertor
8. Gray code to binary convertor

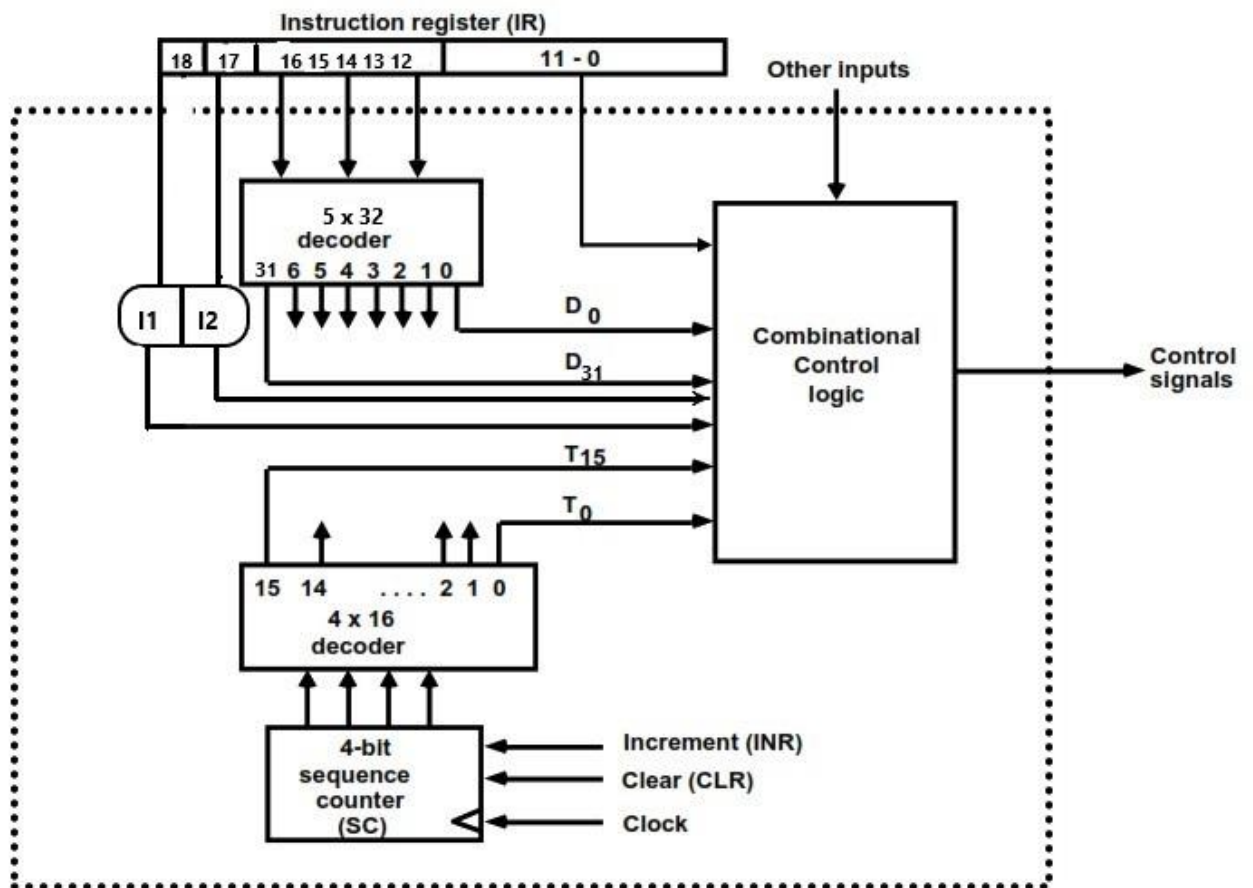


Control unit

Our Control unit might seem similar to a regular one but in reality, it is quite different than its tradition version.

Firstly, we have 2 flip flops, I1 and I2, instead of the conventional I. This allows our control unit to have 4 addressing modes instead of the outdated 2 addressing mode setup.

Another noticeable difference of our Control unit that makes it stand out when compared to the typical control unit is the fact that it has a 5X32 decoder instead of the conventional 3X8 decoder. This is due to the instruction having 5 bits of opcode compared to the standard 4 bits.



Fetch decode cycle

Before the execution of an instruction, the instruction must first be fetched from the computer's memory and passed into the PC. In our processor's case, the R flip flop plays a vital role in determining whether the fetch execute cycle should start, as If and only if R currently has 0 stored in it will the fetch execute cycle be initiated.

The opcode is decoded by a 5X32 decoder to find the instruction's type while the addressing mode part is loaded into the I to determine the addressing mode. (Addressing mode is only checked for memory reference instructions).

The fetching of the instruction happens during 2 clock cycles.

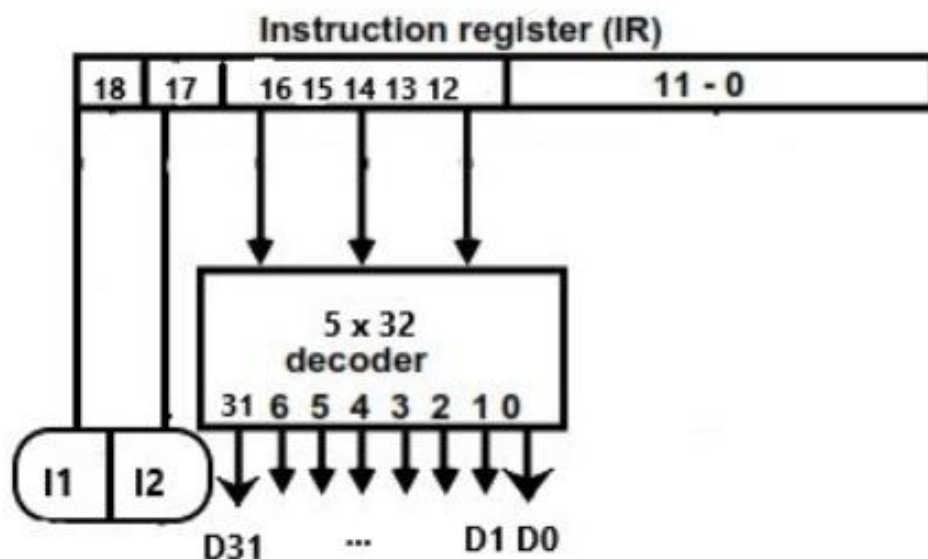
The address in the program counter is transferred to the address register in the first clock cycle.

In the next clock cycle, the instruction contained at the address is fetched and placed in the instruction register, while the program counter is then incremented, now ready for the next instruction.

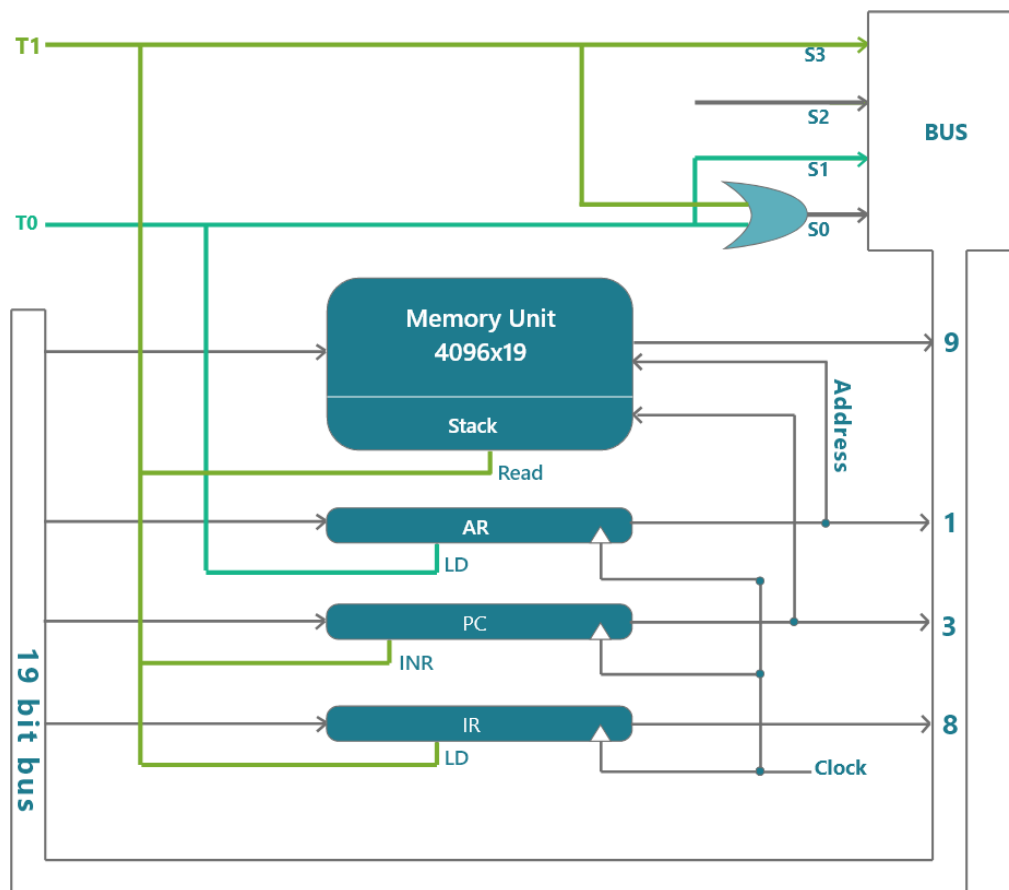
The decoding part has two clock cycles, making it the 3rd and 4th clock cycles of the Fetch and decode cycle.

After decoding and determining which type the current instruction is and checking its addressing mode, in case of a memory reference instruction, the instruction is executed. The sequence counter is then set to 0, and thus this loop is restarted.

Decoding part:



Hardwired T_0 and T_1 timing signals:

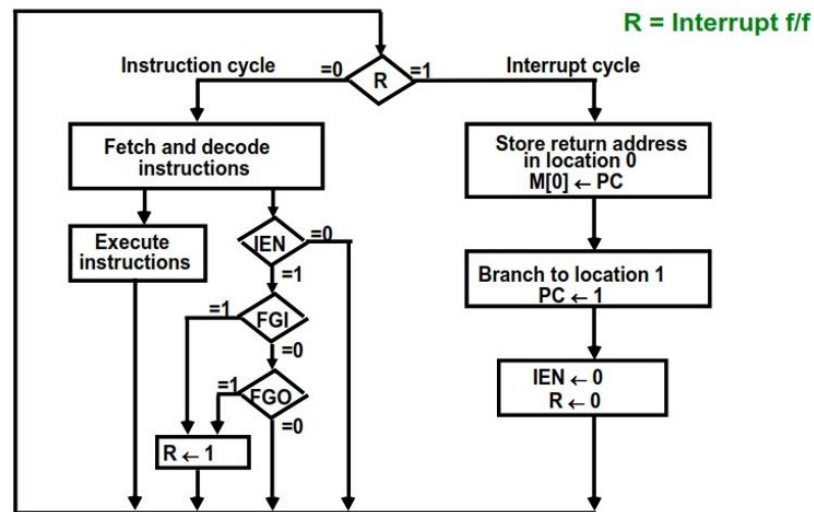


Interrupt Cycle

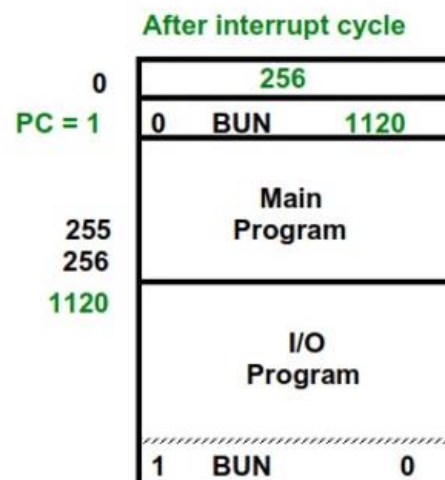
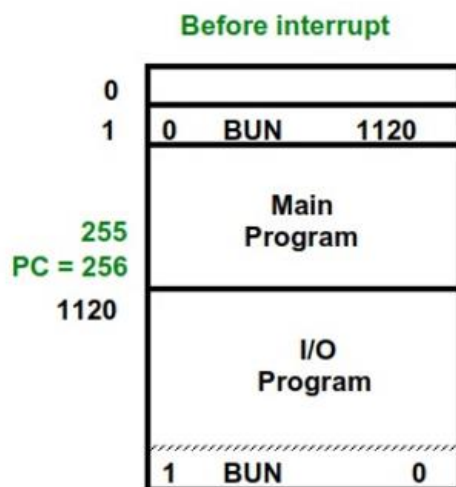
Interrupt is a significant idea in association of PC. It is needed to know whether the program should sit tight for next input or simply continue processing. The interrupt cycle always comes into action whenever the interrupt flag, IEN in our computer, turns on. The program which is currently executing is paused and the return address is stored in memory location 0. The whole interrupt cycle is given below:

- A flip-flop R, known as interrupt flip-flop, is included in the computer.
- If $R=0$, the computer goes through the instruction cycle and if it is 1, it allows computer to start the interrupt cycle.
- There is an IEN flag, known as interrupt enable, which if 1 performs the incoming interrupt cycle, and if 0 ignores the interrupt cycle and continues with the instruction cycle.
- After $IEN=1$, the interrupt cycle checks FGI and FGO which are the incoming input and output flags.
- If one of the two is a 1, that specific register is ready to transfer the information.

- $T0', 1', T2'$ (IEN) (FGI + FGO): $R \leftarrow 1$. This is the control function for initiating the interrupt cycle.
- The whole interrupt cycle is:
 $RT0: AR \leftarrow 0, TR \leftarrow PC$
 $RT1: M[AR] \leftarrow TR, PC \leftarrow 0$
 $RT2: PC \leftarrow PC+1, IEN \leftarrow 0, R \leftarrow 0, SC \leftarrow 0$
- Once the cycle is completed, control checks the value in flip-flop R and decides whether to interrupt or not.



Memory

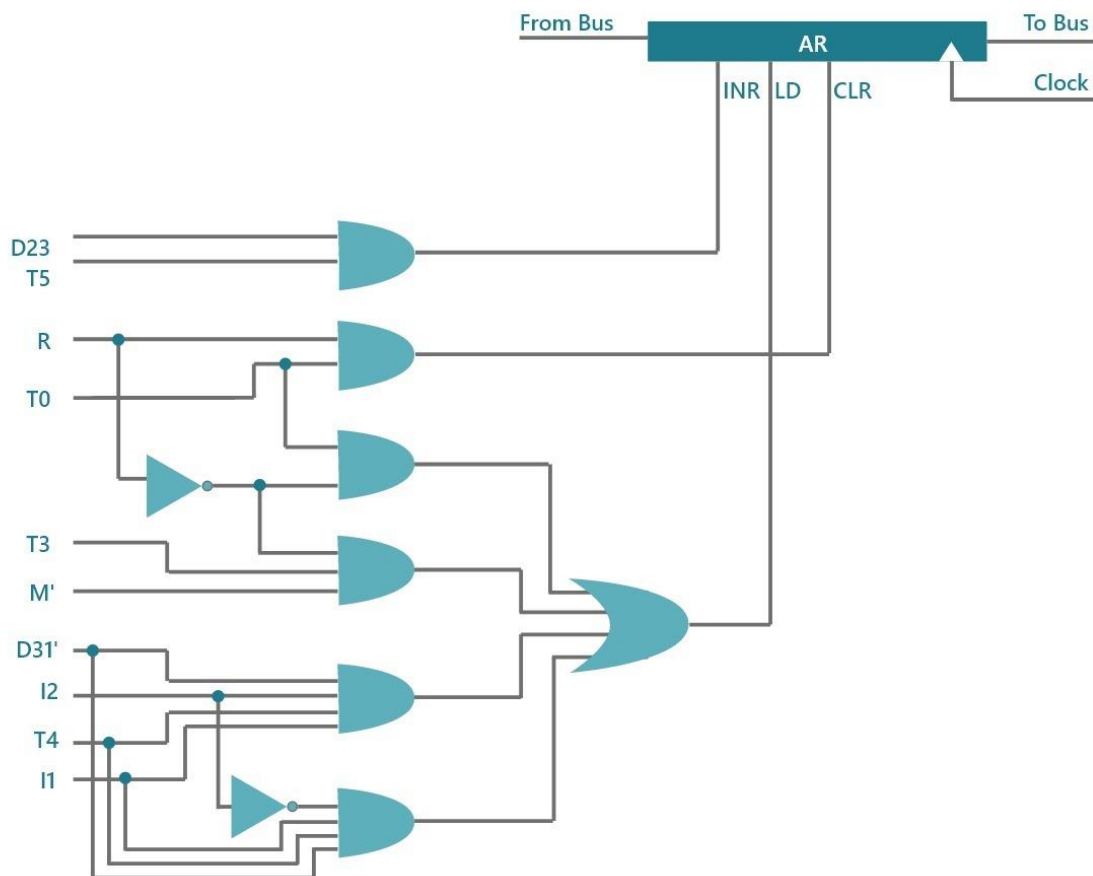


Control of Register

AR:

R' T0 AR <- PC
R' T3 M' AR <- IR(0 ~ 11) (M equals I2I1 when they are 10)
D31' i2'i1 T4 AR <- M[AR]
D31' i2i1 T4 AR <- AR + IX
RT0 AR <- 0 CLR
D23 T5 AR <- AR + 1 INC

INC(AR) = D23 T5
CLR(AR) = R T0
LD(AR) = R' T0 + R' T3 M' + D31' i2'i1 T4 + D31' i2i1 T4



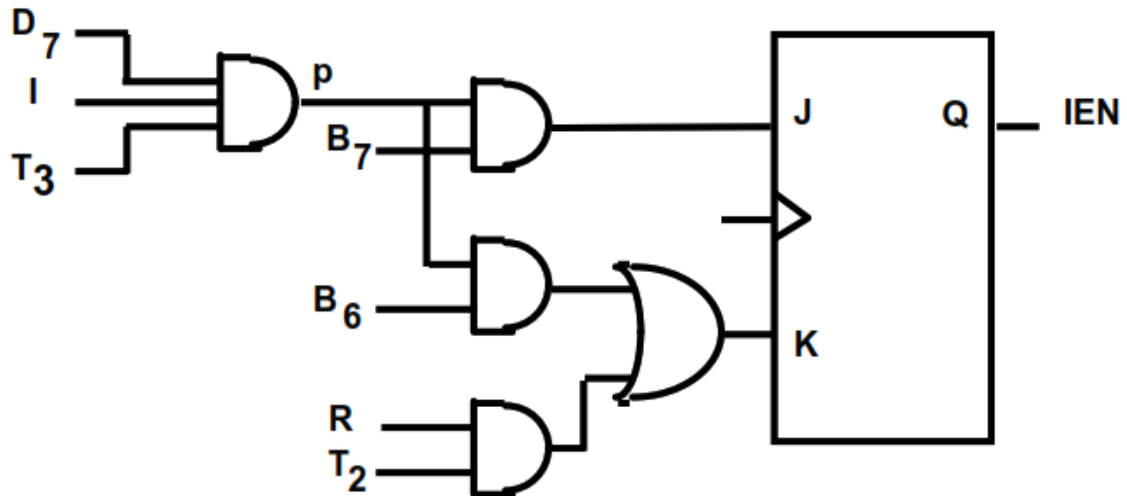
Control of Flag

IEN:

gX_4 : IEN $\leftarrow$ 1 (I/O Instruction)

gX_5 : IEN $\leftarrow$ 1 (I/O Instruction)

RT_4 : IEN $\leftarrow$ 0 (Interrupt)



Control of Common Bus

Selecting AR as source:

D22 T5: PC $\leftarrow$ AR

D23 T6: PC $\leftarrow$ AR

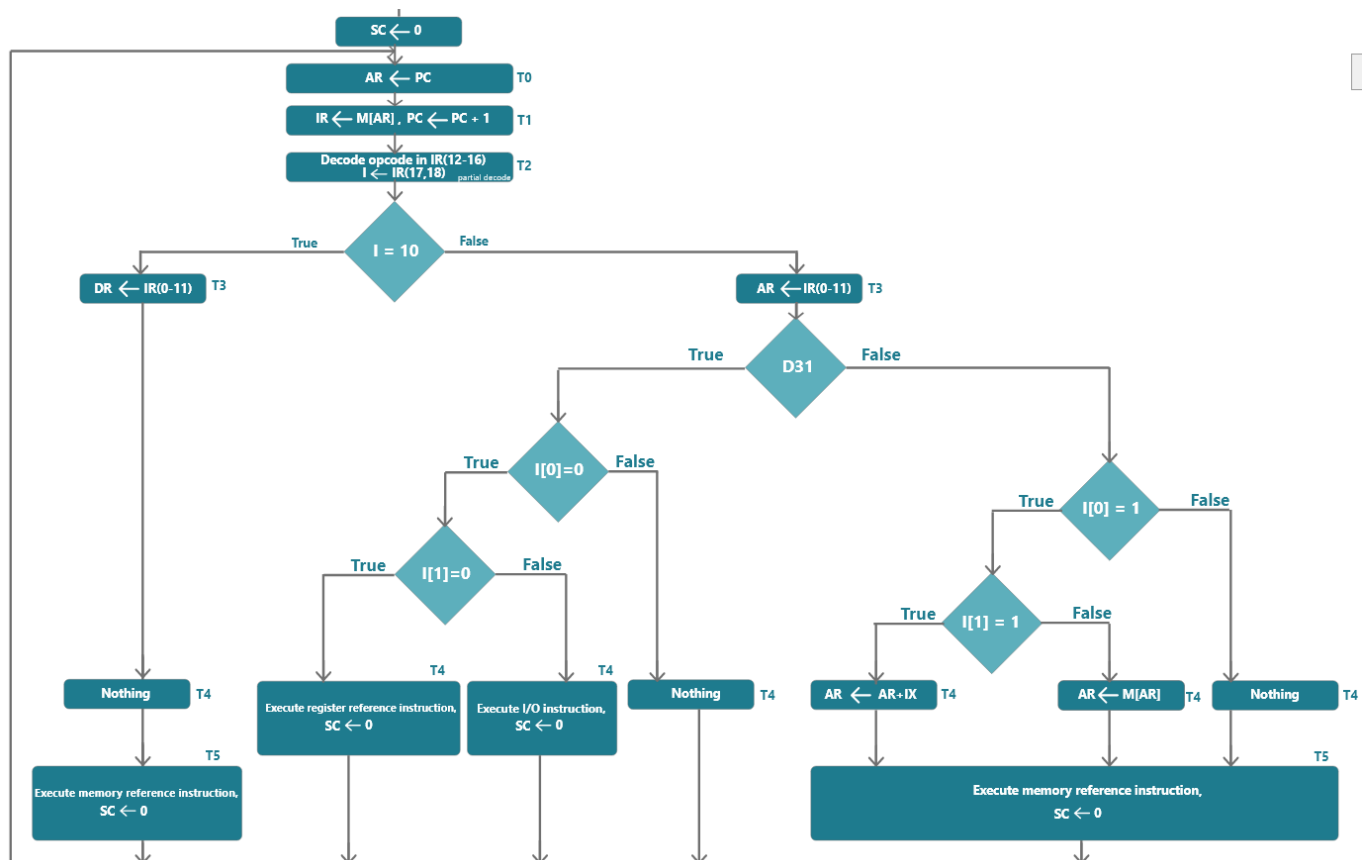
D28 T5: PC $\leftarrow$ AR

D29 T5: PC $\leftarrow$ AR

$$So = D22 T5 + D23 T6 + D28 T5 + D29 T5$$

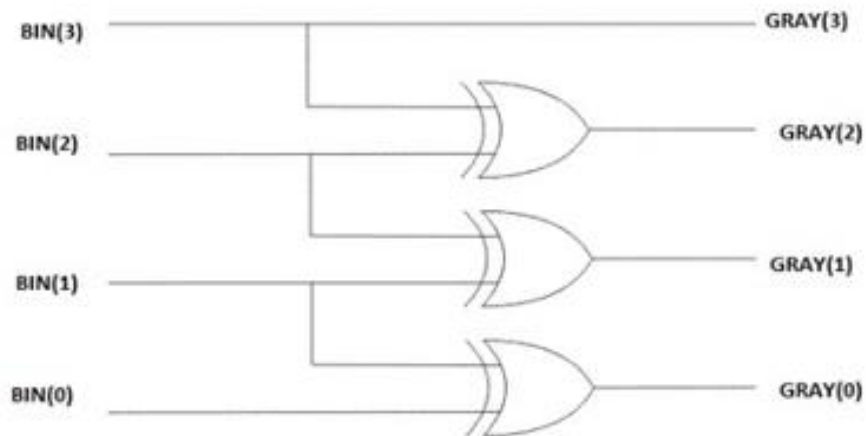
Complete computer description

This following flow chart is complete computer description which is further divided into 3 parts. The first part at T0 and T1 is known as fetching part in which data is fetched from memory. The second part is decoding part which is further divided into 2 parts, firstly at T2 in which opcode and addressing modes are decoded and if the addressing mode is immediate no further decoding takes place at T3 and instruction is just being executed at T5, secondly for other addressing modes at T3 operand is decoded. Furthermore, for execution part it checks the type of instruction it is, either memory reference, register reference or input/output, this is done by decoding the decimal value of opcode which is already done at T2. If the decimal value of opcode is equal to 31 then instruction is register reference or input/output and this is further distinguished by checking the most significant bits whether 00 or 01, for 00 it is register reference and for 01 it is input/output, afterwards for these both type of instructions at T4 instruction is executed. Moreover, if decimal value of opcode is less than 31 then it is memory reference instruction and these are further related with addressing modes which are either direct, indirect, or indexed, and each of these have some operations to be performed which are done at T4, after this at T5 execution is done. Finally, sequence counter is set to 0 at the end of all these executions and next instruction is fetched.

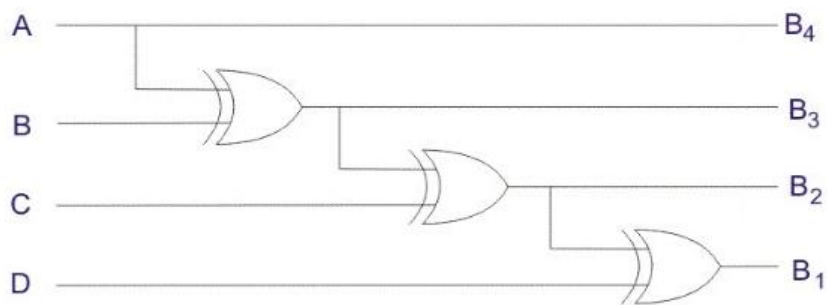


ALU Circuits

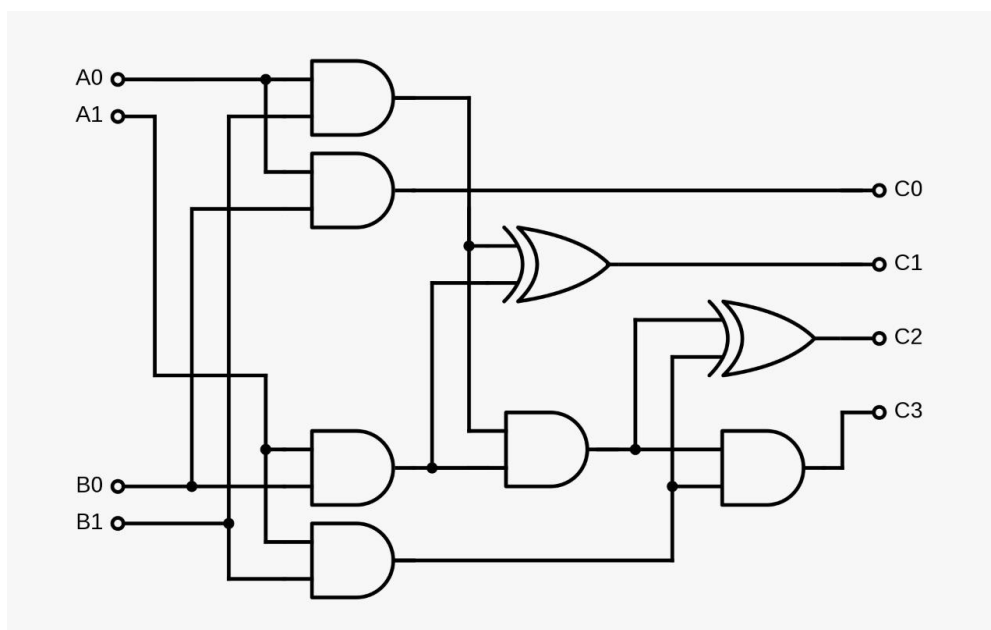
Binary To Gray Code Circuit:



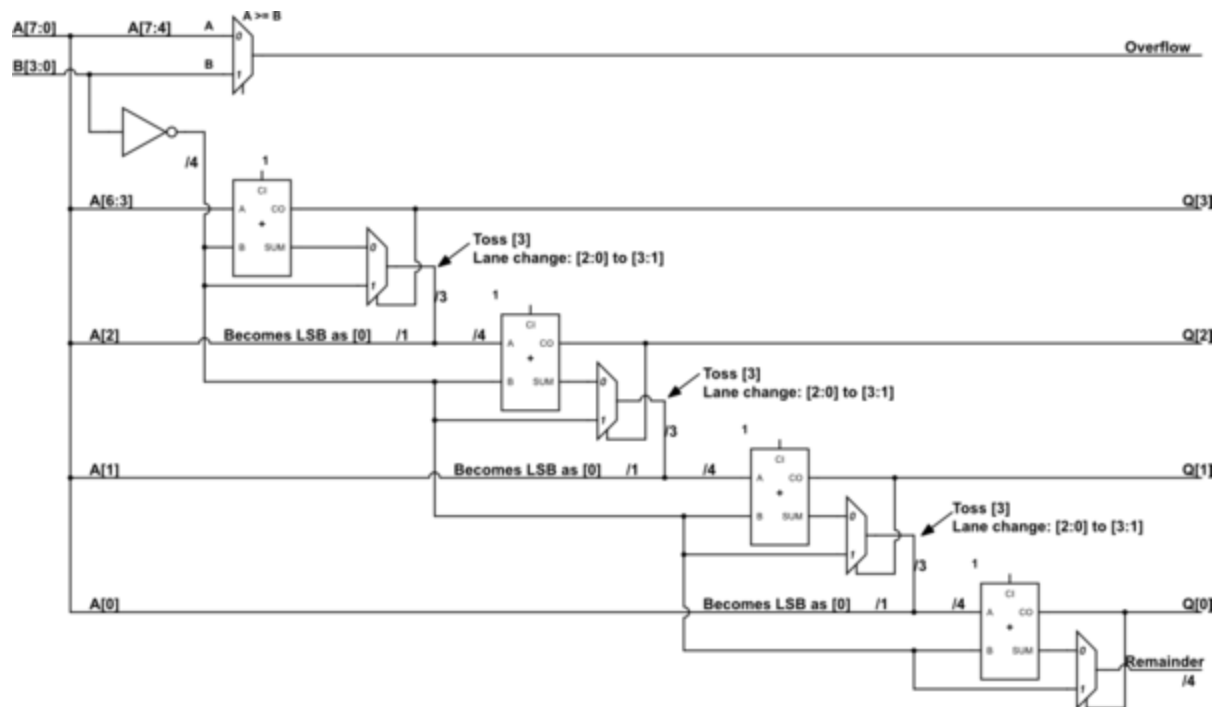
Binary To Gray Code Circuit:



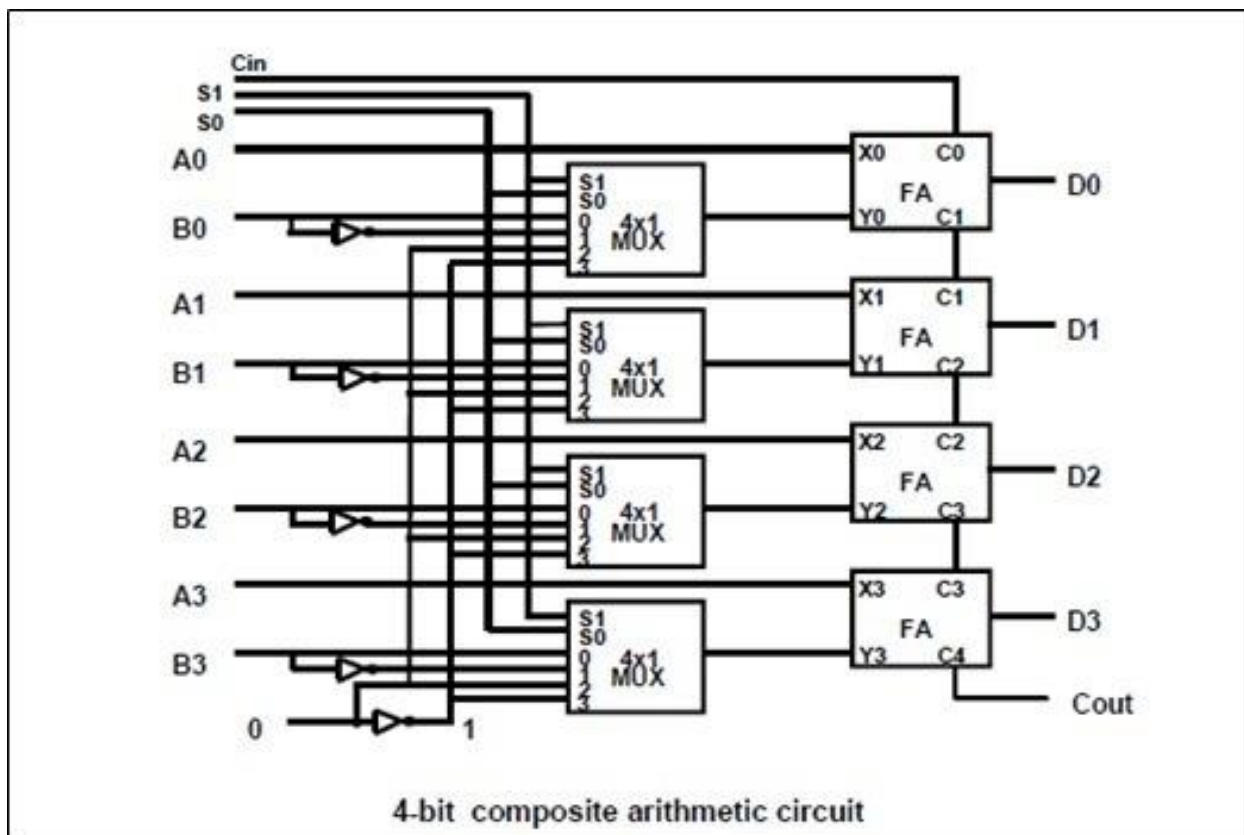
Multiplier:



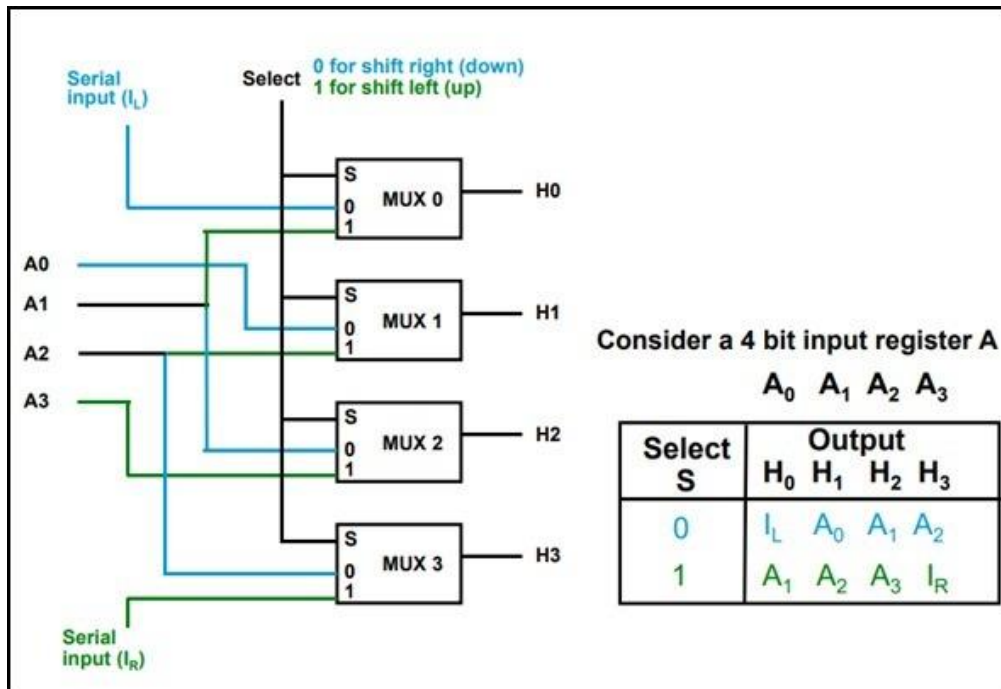
Divide Logic circuit:



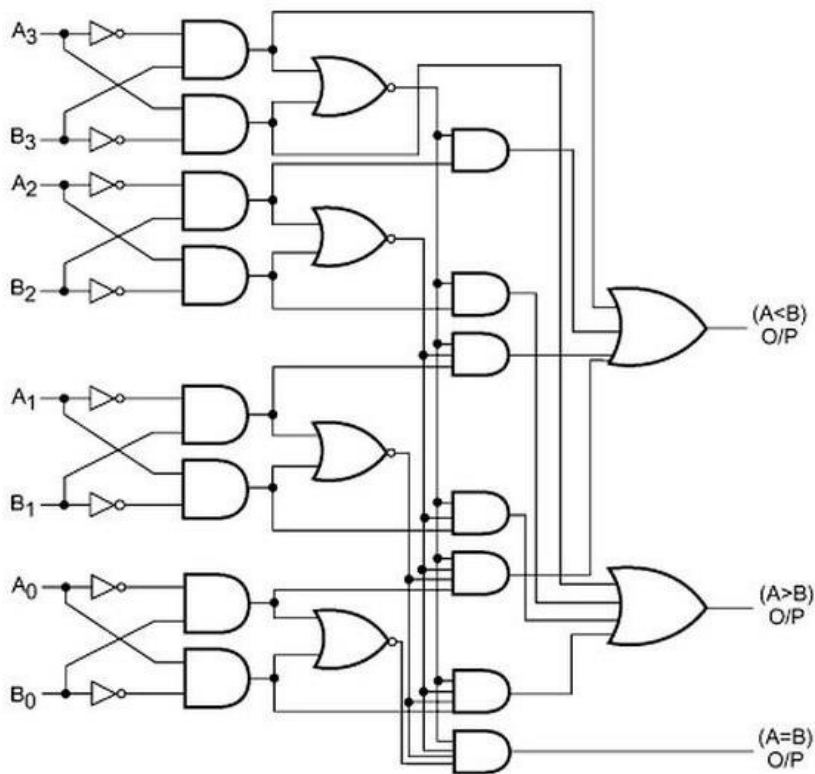
Arithmetic Circuit:



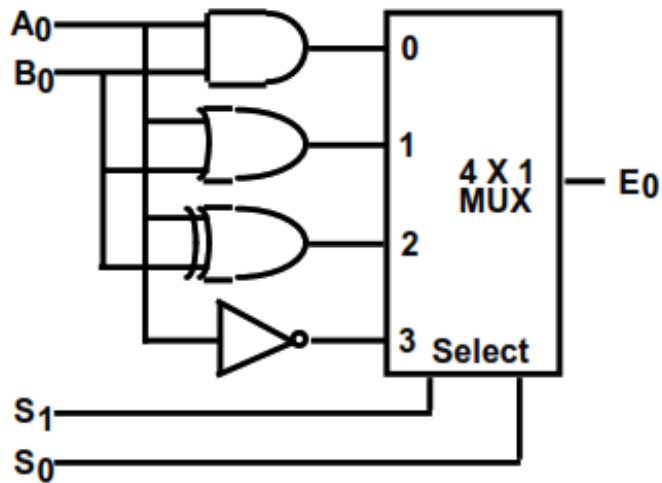
Logical Shift Circuit:



Comparator:



Logic Circuit:



Bibliography

- computer system architecture book by m. morris mano
- CS-222 lecture slides
https://upload.wikimedia.org/wikipedia/commons/thumb/3/34/Binary_multiplier.svg/500px-Binary_multiplier.svg.png
- https://www.google.com/search?q=divider+logic+circuit&tbm=isch&ved=2ahUK_EwiztoOu7PHwAhWR_4UKHSM0BNIQ2-cCegQIABAA&oq=divider+logic+c&gs_lcp=CgNpbWcQARgAMgIIADIGCAAQBRAeMgYIABAFEB4yBggAEAgQHjIGCAAQCBAeOgQIABBDOgQIABAeUJAvWKRAYORlaABwAHgAgAGHAogBvw6SAQMyLTiYACgAQGgAQtn3Mtd2l6LWltZ8ABAQ&sclien t=img&ei=FsCzYLPTBJH_lwSj6JCQDQ&bih=937&biw=1920#imgsrc=nKoHoUiBs_1hdM